

Multi-Level Replication (Cascading Replication / Hierarchical Replication)

1. What is Multi-Level Replication?

Multi-level replication is an extension of single-leader replication where replicas themselves replicate data to other replicas.

None

Leader

└─ Replica A

| └─ Replica A1

| └─ Replica A2

└─ Replica B

 └─ Replica B1

 └─ Replica B2

Also called:

- Cascading replication
- Tiered replication
- Tree replication

2. Why It Is Needed

In normal single-leader replication:

None

Leader → all followers

Leader has limits:

- CPU
- Network bandwidth
- Connections
- Disk throughput

If thousands of replicas depend directly on leader, bottleneck occurs.

So use hierarchy.

3. Core Idea

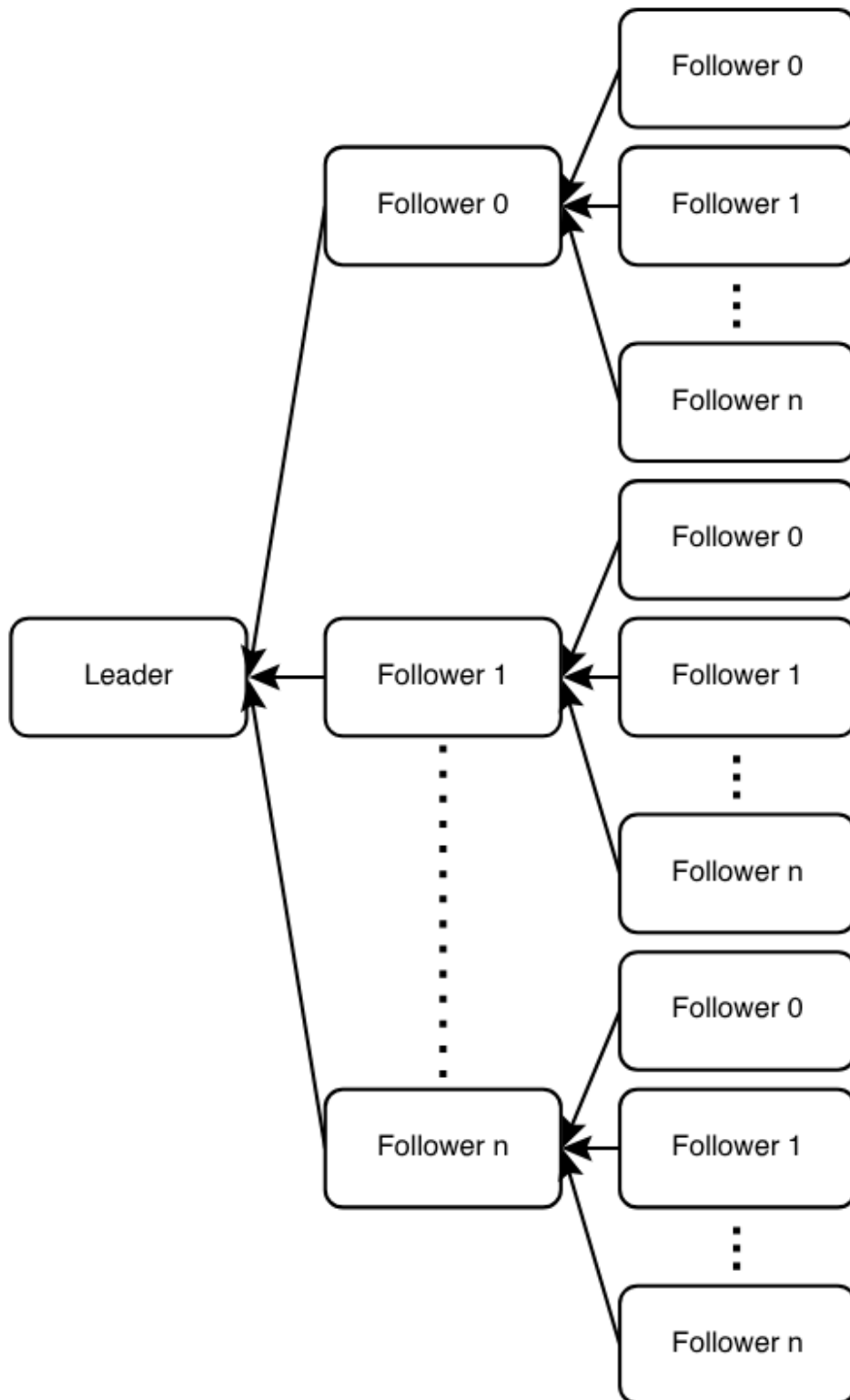
Each node only replicates to a manageable number of children.

None

Leader sends to 5 replicas

Each replica sends to 5 more

This dramatically increases fan-out.



4. Example

Instead of leader pushing to 100 replicas directly:

None

1 → 100

Use:

None

1 → 10

10 → 100

Much easier on leader.

5. Main Benefit = Massive Read Scaling

You can support many read replicas globally.

Useful for:

- Large content platforms
- Reporting systems
- Read-heavy SaaS
- Global catalogs

6. Geographic Example

None

Primary: Mumbai

Level 1 Replica: Singapore

Level 2 Replica: Tokyo / Sydney / Dubai

Regional replicas then serve nearby users.

Lower latency.

7. Trade-off = More Replication Delay

Important downside:

None

Leader write

↓

Replica A after 50 ms

↓

Replica A1 after 120 ms

Lower levels may be more stale.

This increases:

- Eventual consistency delay
- Replica lag
- Read freshness issues

8. Failure Impact

If intermediate node fails:

None

Children replicas lose upstream updates

Need re-parenting or rerouting.

9. Comparison with Single-Level Replication

Feature	Single-Level	Multi-Level
Leader load	High	Lower
Replica count supported	Limited	Very high
Replication lag	Lower	Higher
Complexity	Low	Higher

10. Real World Examples

- MySQL cascading replicas
- PostgreSQL cascading replication
- YouTube style global read distribution concepts
- Netflix regional data distribution patterns

11. MySQL Binlog Replication

MySQL often uses binary logs (binlogs):

None

Leader writes binlog

Replica reads and replays changes

Replicas may further feed downstream replicas.

12. Main Limitation Mentioned

Even with replication:

None

Entire writable dataset still tied to one leader

If one machine cannot hold/write entire DB:

Need:

- Sharding
- Partitioning
- Multi-leader
- Distributed write systems

13. When to Use

Choose multi-level replication when:

None

Huge read traffic

Many regions

Leader overloaded by replicas

Read freshness less critical

14. When to Avoid

Avoid when:

None

Need latest reads everywhere

Frequent writes

Low-latency strong consistency required

15. Interview Answer Example

Q: How do you scale read replicas beyond leader limits?

Use cascading/multi-level replication where leader replicates to first-tier replicas, which then replicate further downstream.

16. Short Revision Notes

None

Multi-Level Replication:

Leader → Replicas → More Replicas

Pros:

massive read scaling

lower leader load

Cons:

more lag

more complexity

17. One-Line Summary

Multi-level replication scales read traffic beyond leader limits by using hierarchical replicas, trading simplicity and freshness for massive fan-out capacity.